

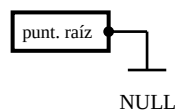
ARBOL BINARIO

Un árbol es una estructura de datos que va creciendo en profundidad desde un nodo de comienzo llamado raíz y se expande a diversos nodos a los cuales se accede partiendo siempre desde la raíz. A cada uno de estos nodos se los llama rama ya su vez cada rama apunta a otros nodos que se llaman hojas, si estos no apuntan a otro nodo, pero si así lo hacen pasan a ser nuevas ramas. Un árbol esta subdividido en subárboles y así sucesivamente, cada subárbol es un árbol en si mismo.

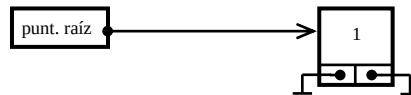
Como caso particular del árbol esta el llamado árbol binario, vamos a desarrollar este tipo particular en estas páginas. El árbol binario tiene la particularidad de que cada nodo tiene solo dos nodos que lo suceden uno llamado derecho y el otro izquierdo.

Cada nodo de este tipo tiene dos punteros (derecho e izquierdo) que apuntan a los nodos que le siguen.

Para comenzar a crearlo se utiliza un puntero a la raíz del mismo la cual apunta a un valor NULL.



Cuando se genera el nodo raíz el puntero a raíz apuntara al primer nodo del árbol, los dos punteros de este apuntaran a NULL.

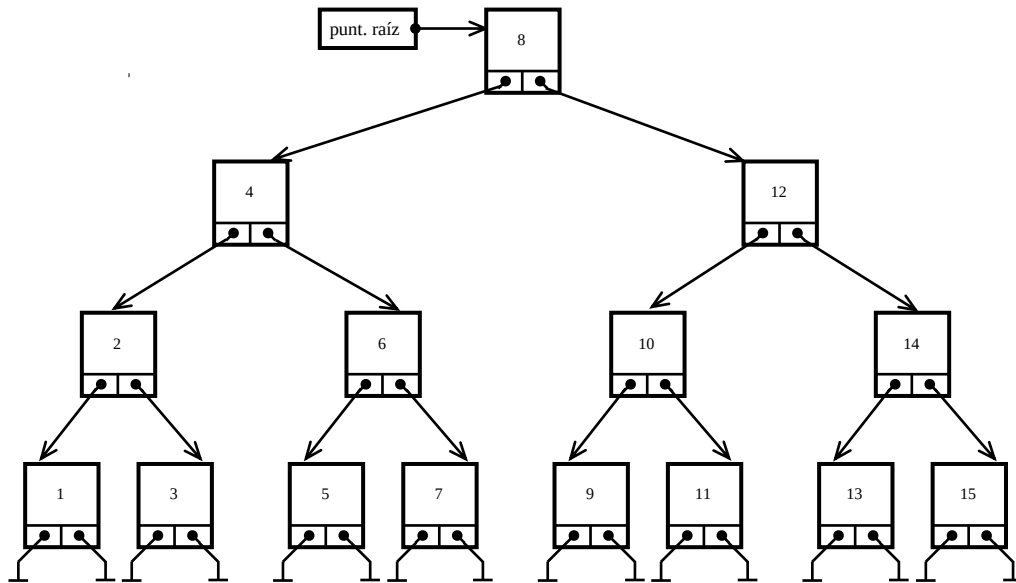


El segundo elemento se insertará en la posición correspondiente para poder obtener un árbol ordenado. El orden del árbol se obtiene a partir de cómo se lo recorre, estos pueden ser de tres tipos diferentes.

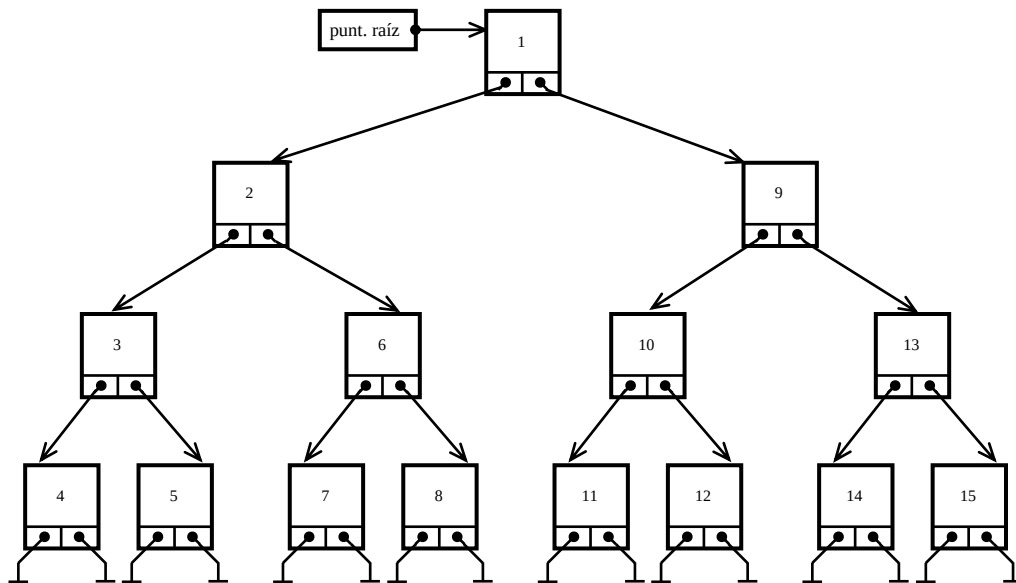
- Inorden
- Preorden
- Postorden

Estos recorridos se basan en la forma en que se recorre al árbol y los datos quedan ubicados según el tipo de camino elegido.

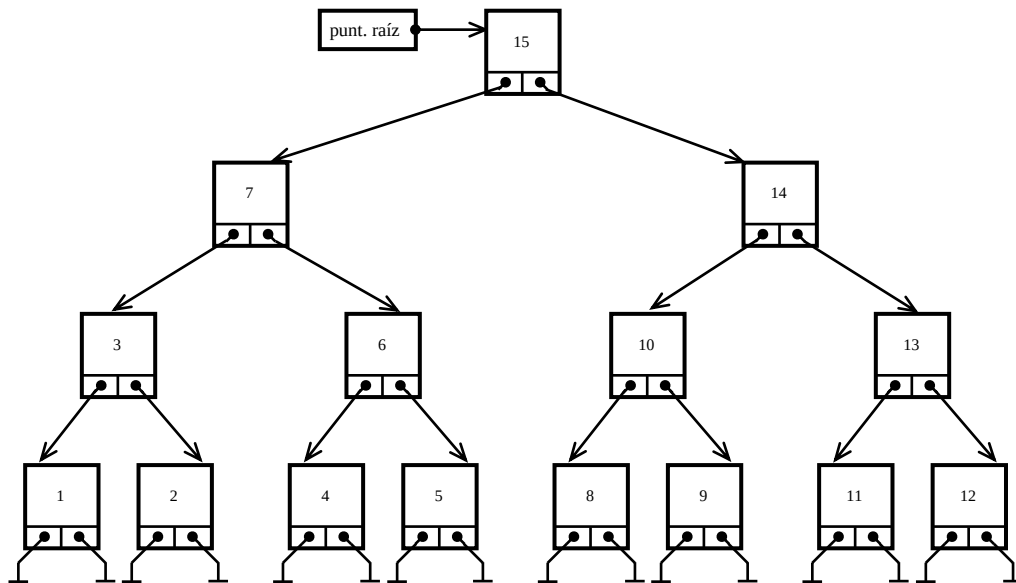
El recorrido inorden se basa en comenzar a viajar por el árbol comenzando con el subárbol izquierdo, luego la raíz y por ultimo el subárbol derecho.



Se puede observar en la figura el orden de los datos según el recorrido inorden. En el tipo de camino preorden se recorre al árbol comenzando desde la raíz, prosiguiendo por el subárbol izquierdo y luego terminando por el subárbol derecho.



Se puede observar en la figura el orden de los datos según el recorrido preorden. Si se realiza un recorrido postorden se visita primero el subárbol izquierdo, el subárbol derecho y al final la raíz.



Se puede observar en la figura el orden de los datos según el recorrido postorden. Para poder listar un árbol hay que realizarlo según el tipo de recorrido elegido. Lo mismo ocurre cuando se quiere buscar algún elemento en él. Si se debe borrar algún elemento se debe realizar una reestructuración del árbol según el camino elegido para construirlo. En este tipo de estructura de datos los algoritmos deben ser de tipo recursivo, para poder hacerlos lo más simple posible. Si se tratara de encontrar un camino que no fuese de esa forma los algoritmos serían mucho más complicados.

Ejemplo

```

1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #include <conio.h>
4.  struct arb { int a;
5.              struct arb *iz;
6.              struct arb *de;};
7.  void ingresa(struct arb **);
8.  void lee(struct arb *,int);
9.  void busca(struct arb *);
10. struct arb *enc_dat(struct arb *,int);
11. void borra(struct arb **);
12. struct arb *bor(struct arb *,int);
13. struct arb *ing_dat(void);
14. struct arb *inord(struct arb *,struct arb *);
15. void main(void)
16. {
17.     struct arb *raiz=NULL;
18.     int op;
19.     for(;;)
20.     {
21.         clrscr();
22.         printf("1 - Ingresar datos\n");
23.         printf("2 - Leer datos\n");
  
```

```

24.     printf("3 - Borrar datos\n");
25.     printf("4 - Buscar datos\n");
26.     printf("5 - Salir\n\n");
27.     printf("Ingrese una opcion ( 1 - 5 ) : ");
28.     fflush(stdin);
29.     scanf("%d",&op);
30.     switch(op)
31.     {
32.         case1: ingresa(&raiz);
33.             break;
34.         case2: lee(raiz,0);
35.             printf("\n\nPresione una tecla para");
36.             printf("continuar\n");
37.             getch();
38.             break;
39.         case3: borra(&raiz);
40.             break;
41.         case4: busca(raiz);
42.             break;
43.         case5: exit(0);
44.     }
45. }
46. }
47. void ingresa(struct arb **act)
48. {
49.     char k;
50.     struct arb *nvo;
51.     clrscr();
52.     do
53.     {
54.         nvo=ing_dat();
55.         if(!nvo)
56.             return;
57.         else
58.             *act=inord(*act,nvo);
59.         printf("\nOtro dato ( S / N )? ");
60.         k=toupper(getche());
61.     }
62.     while(k!='S');
63. }
64. struct arb *ing_dat(void)
65. {
66.     struct arb *nvo;
67.     nvo=malloc(sizeof(struct arb));
68.     if(nvo)
69.     {
70.         printf("\n\nIngrese el dato : ");
71.         scanf("%d",&nvo->a);
72.         nvo->iz=NULL;
73.         nvo->de=NULL;
74.     }
75.     else
76.     {
77.         clrscr();
78.         printf("\n\nNo hay memoria disponible\n");
79.         printf("\n\nPresione una tecla para continuar\n");
80.         getch();
81.         return(NULL);
82.     }

```

```

83.     return(nvo);
84. }
85. struct arb *inord(struct arb* act,struct arb* aux)
86. {
87.     if(!act)
88.         act=aux;
89.     else if(act->a<aux->a)
90.         act->de=inord(act->de,aux);
91.     else
92.         act->iz=inord(act->iz,aux);
93.     return(act);
94. }
95. void lee(struct arb *raiz,int a)
96. {
97.     int i;
98.     if(raiz->iz)
99.         lee(raiz->iz,a+1);
100.    else
101.    {
102.        for(i=0;i<a;i++)
103.            printf("\t");
104.        printf("%d\n",raiz->a);
105.        if(raiz->de&&!raiz->iz)
106.            lee(raiz->de,a+1);
107.        return;
108.    }
109.    for(i=0;i<a;i++)
110.        printf("\t");
111.    printf("%d\n",raiz->a);
112.    if(raiz->de)
113.        lee(raiz->de,a+1);
114.    else
115.        return;
116. }
117. void busca(struct arb *root)
118. {
119.     int i;
120.     char k;
121.     struct arb *raiz;
122.     clrscr();
123.     do
124.     {
125.         raiz=root;
126.         printf("ingrese el dato :");
127.         scanf("%d",&i);
128.         raiz=enc_dat(raiz,i);
129.         if(raiz)
130.             printf("\n\nEl dato buscado es :\t%d\n",raiz->a);
131.         else
132.         {
133.             printf("\n\nEl dato no ha sido encontrado");
134.             printf("\n\nPresione una tecla para continuar\n");
135.             getch();
136.         }
137.         printf("\nOtro dato ( S / N )? ");
138.         k=toupper(getche());
139.     }
140.     while(k!='S');

```

```

141. }
142. struct arb *enc_dat(struct arb *raiz,int i)
143. {
144.     if(!raiz)
145.         return(NULL);
146.     if(raiz->a==i)
147.         return(raiz);
148.     if(raiz->a>i)
149.         if(raiz->iz)
150.             raiz=enc_dat(raiz->iz,i);
151.         else
152.             return(NULL);
153.     elseif(raiz->de)
154.         raiz=enc_dat(raiz->de,i);
155.     else
156.         return(NULL);
157. }
158. void borra(struct arb **root)
159. {
160.     int i;
161.     char k;
162.     struct arb *raiz;
163.     do
164.     {
165.         raiz=*root;
166.         printf("\n\ningrese el dato a borrar : ");
167.         scanf("%d",&i);
168.         raiz=bor(raiz,i);
169.         if(raiz)
170.             *root=raiz;
171.         else
172.         {
173.             printf("\n\nEl dato no ha sido encontrado");
174.             printf("\n\nPresione una tecla para continuar\n\n");
175.             getch();
176.         }
177.         printf("\n\nOtro dato ( S / N )? ");
178.         k=toupper(getche());
179.     }
180.     while(k!='S');
181. }
182. struct arb *bor(struct arb *raiz,int i)
183. {
184.     struct arb *p,*p2;
185.     if(!raiz)
186.         return(NULL);
187.     if(raiz->a==i)
188.     {
189.         if(raiz->iz==raiz->de)
190.         {
191.             free(raiz);
192.             return(NULL);
193.         }
194.         elseif(!raiz->iz)
195.         {
196.             p=raiz->de;
197.             free(raiz);
198.             return(p);

```

```

199.         }
200.         else if(!raiz->de)
201.         {
202.             p=raiz->iz;
203.             free(raiz);
204.             return(p);
205.         }
206.         else
207.         {
208.             p2=raiz->de;
209.             p=raiz->de;
210.             while(p->iz)
211.                 p=p->iz;
212.             p->iz=raiz->iz;
213.             free(raiz);
214.             return(p2);
215.         }
216.     }
217.     if(raiz->a<i)
218.         raiz->de=bor(raiz->de,i);
219.     else
220.         raiz->iz=bor(raiz->iz,i);
221.     return(raiz);
222. }

```

En el ejemplo se detalla un programa que permite el ingreso, la lectura, la búsqueda y el borrado de un elemento de un árbol. El recorrido de este es del tipo inorden.

Cada elemento del árbol esta contenido en una estructura del tipo arb la que se define entre las líneas 4 y 6, esta tiene tres campos uno de tipo int (a) en el que se va a guardar el dato y dos punteros a la estructura (de e iz) los que serán los que se encargaran de apuntar a las ramas siguientes del árbol.

La función main se define entre las líneas 15 y 46. Esta se encarga de generar un menú de opciones e invocar a las funciones correspondientes. Tiene un lazo for para recorrer el menú hasta que se elija la opción de salir, en tal caso se invocara a la función exit con el parámetro 0, indicando el fin normal del programa.

Cuando se elige la opción ingresar datos se invoca a la función ingresa (líneas 47 a 63). A esta se le pasa la dirección del puntero que apunta a la raíz del árbol en un puntero a puntero a struct arb llamado act y devuelve void. Su prototipo esta en la línea 7.

Permite ingresar un dato en la posición correspondiente en el árbol. Esta inmersa en un lazo do-while para permitir el ingreso de mas de un valor cada vez que se la invoque.

En la línea 54 se invoca a la función ing_dat, que devuelve el valor de la posición de memoria en donde se ubica la posición del bloque en donde estará el nuevo elemento. En el caso de que el valor retornado sea NULL, se retornara a la función main (línea 56). Si ocurre lo contrario se invoca a la función inord (línea 58) a la cual se le pasan los valores de la dirección de la raíz del árbol y la dirección del nuevo elemento.

La función ing_dat (líneas 64 a 84) permite solicitar memoria e ingresar el nuevo dato. Recibe void como parámetro y devuelve un puntero a struct arb, en donde se retornará el valor de la posición de memoria del bloque del nuevo dato. El prototipo de la misma está ubicado en la línea 13.

En la línea 67 se solicita la memoria necesaria para el nuevo dato mediante la invocación de la función malloc. Si malloc devuelve NULL se informará tal

circunstancia y se retornará a ingresa un valor NULL (líneas 77 a 81). De no ser así se ingresa el valor del dato y se le asignan a los punteros de e iz el valor NULL (líneas 70 a 73). En la línea 83 se retorna la dirección de memoria del nuevo elemento.

La función inord (líneas 85 a 94) es una función recursiva que permite ubicar al nuevo elemento del árbol en el lugar correspondiente, a esta se le pasan dos punteros a struct arb (act y aux) que contendrán los valores de los punteros a inicio y del nuevo elemento respectivamente, devuelve un puntero a struct arb que tendrá la nueva posición de inicio. Su prototipo esta ubicado en la línea 14.

Si el valor del puntero act es NULL quiere decir que no existe una rama que le sigue y por lo tanto hay que ubicar al nuevo elemento en dicho lugar, esto se hace asignándole el valor del nuevo elemento que esta contenido en aux (líneas 87 y 88).

En caso contrario se pregunta si el valor del dato al que apunta act es menor al valor del dato al que apunta aux (línea 89), de ser así se debe examinar el subárbol derecho llamando recursivamente a inord, pero enviándole los parámetros act->de y aux y retornando el valor al puntero act->de (líneas 90). Si es mayor se debe recorrer el subárbol izquierdo de la misma forma que en el otro caso pero enviándole act->iz y aux y retornado en act->iz (líneas 92).

En la línea 93 se retorna el valor de act que actualizará el valor del puntero correspondiente.

La función lee (líneas 95 a 116) permite imprimir en pantalla al árbol. Su prototipo se ubica en la línea 8. Recibe un puntero a struct arb que indicará la dirección del elemento a imprimir y un int que será el desplazamiento en pantalla de dicho elemento; devuelve void.

Es una función recursiva. Busca un puntero NULL en el subárbol izquierdo, cuando lo encuentra imprime el valor de dicho elemento (líneas 98 a 104). Luego pregunta si hay algún valor en el subárbol derecho y continua buscándolo hasta encontrar el NULL y luego lo imprime (líneas 105 a 107), retornando a la función que lo invocó. Imprime la raíz (líneas 109 a 111) y luego busca en el subárbol derecho hasta encontrar un NULL y retornar (líneas 112 a 115).

La función busca (líneas 117 a 141) permite buscar un elemento dado en un árbol e imprimirlo en la pantalla. Recibe un puntero a struct arb (root), en él se envía el valor del puntero a la raíz del árbol y devuelve void. Esta en un lazo do-while para permitir más de un proceso cada vez que se la invoque. Se inicializa al puntero raíz con el valor de root, para comenzar la búsqueda dentro del arbol, luego se ingresa el vaslor a encontrar y se llama a la función enc_dat pasándosele el valor de raíz y el elemento a buscar, devuelve la dirección del elemento que lo contiene o NULL en caso de no haberlo encontrado (líneas 125 a 128).

Si el valor buscado ha sido encontrado se lo imprime (líneas 129 y 130). Cuando esto no sucede se indica tal situación (líneas 133 a 135).

La función enc_dat (líneas 142 a 157) recibe un puntero a struct arb (raíz), donde se ubica la dirección del elemento desde donde se comienza a buscar, y un int (i), donde se guarda el valor a buscar, devuelve un puntero a struct arb con el valor del puntero del elemento encontrado o NULL. El prototipo se ubica en la línea 10. Es una función recursiva.

Primero se pregunta si el valor de raíz es NULL en cuyo caso se lo retornará (líneas 144 y 145).

A continuación se verifica si el valor del árbol es el buscado y en dicho caso se retornará éste (líneas 146 y 147).

En caso de no haberlo encontrado se pregunta si es mayor que el buscado, preguntando si el puntero del subárbol izquierdo es distinto a NULL, de ser así se lo sigue buscando en él, caso contrario se retornara NULL (líneas 148 a 152).

Si es menor se lo buscará en el subárbol derecho, siempre y cuando el puntero a este subárbol sea distinto de NULL, si es así se devuelve NULL (líneas 153 a 156).

La función borra (líneas 158 a 181) permite borrar uno o mas elementos de un árbol ya que esta dentro de un lazo do_while, recibe un puntero a struct arb (root), en donde recibe el valor de la dirección del puntero a la raíz del árbol y devuelve void. Su prototipo esta en la línea 11.

Se inicializa al puntero raiz con el valor recibido, se ingresa el valor del elemento a borrar, y se invoca a la función bor pasándosele este puntero y el valor a borrar; devolverá el valor a raiz, en caso de que haya sido modificado se lo actualizará, si no se devolverá el mismo valor (líneas 165 a 168).

Se evalúa el valor de raiz, si este es distinto de NULL se actualiza *root, si no se informa que no fue encontrado (líneas 169 a 176).

La función bor (líneas 182 a 222) es la encargada de encontrar el lugar en donde se ubica el elemento a borrar y modificar el árbol debido a tal evento, recibe un puntero a struct arb raiz, en donde se recibe el valor de la raíz del árbol y un int i en donde recibe el valor a buscar, devuelve un puntero a struct arb. Su prototipo esta en la línea 12. Es una funcion recursiva.

Se evalúa si raiz es NULL en cuyo caso se retorna este valor debido a que no ha sido encontrado (líneas 185 y 186).

En la línea 187 se evalúa si el valor ha sido encontrado, en tal caso se evalúa si los punteros de e iz son iguales, se libera la memoria y se retorna NULL, indicando que es el último elemento del árbol (líneas 189 a 193). Si no ocurre esta situación se evalúa si raiz->iz es NULL se asigna al puntero p el valor de raiz->de es NULL, se libera la memoria y se retorna el valor del puntero p (líneas 194 a 199). Si raiz->iz es distinto a NULL se evalúa si raiz->de es NULL y se procede de la misma manera que con raiz->iz (líneas 200 a 205). En el caso de que no ocurra ninguna de las situaciones anteriormente explicadas se procede a asignarle a los punteros p y p2 el valor de raiz->de, se ingresa a un ciclo while con la condición de que p->iz sea distinto de NULL y en él se actualiza a p con el valor p->iz, se libera la memoria y luego se retorna el valor de p2 (líneas 208 a 214).

Si el valor no se encontró se evalúa si el valor del árbol es menor al buscado, en este caso se invoca a bor con los parámetros raiz->de e i y el valor retornado se pasa a raiz->de, si es mayor se llama a si misma con los parámetros raiz->iz e i y su retorno se hará en raiz->iz (líneas 217 a 220). Por último se retorna el valor del puntero raiz (línea 221).